

Python for Security

Unit 06 — Technical Foundations

Last week you scripted in Bash. This week, same ideas — variables, loops, functions — in a language real security tools are written in.

Learning objectives

By the end of this unit you can:

- **Explain** why Python is popular in security.
- **Run** Python two ways: interactive (`python3`) and a script (`python3 file.py`).
- **Use** variables and **types** (`str`, `int`, `float`, `bool`); convert between them.
- **Work with** strings, lists, and dictionaries.
- **Write** `if` / `elif` / `else` and `for` / `while` loops.
- **Define** a function with parameters and a `return`.
- **Import** a module (`socket`) and use `try` / `except`.
- **Build** a working **TCP port scanner** for the isolated lab target.

Why Python?

- **Readable** — looks almost like English.
- **Huge library ecosystem** — networking, web, crypto, all built in.
- Real tools and exploit proof-of-concepts are written in it (Impacket, many PoCs).
- **Cross-platform** — same code on Kali, macOS, Windows.

Bash glues commands together; Python builds the **tools** themselves.

Two ways to run Python

Interactive (REPL) — type one line, see the result instantly:

```
python3
>>> 2 + 2
4
>>> print("hello")
hello
>>> exit()
```

Script — a `.py` file that runs top to bottom:

```
python3 hello.py
```

Python is not Bash

	Bash	Python
Variable use	<code>\$name</code>	<code>name</code>
Print	<code>echo</code>	<code>print(...)</code>
Indentation	cosmetic	syntax!
Version	—	use <code>python3</code>

 Indentation **is the code** in Python. Mixed tabs/spaces → `IndentationError`. Pick spaces and stick with it.

Variables & types

```
name = "Jordan"      # str  (text)
port = 22             # int  (whole number)
timeout = 0.5         # float (decimal)
is_open = True        # bool (True / False)

print(type(port))    # <class 'int'>
```

Convert between them with `int()` and `str()`:

```
int("22") + 1        # 23
str(22) + "1"         # "221"
```

Strings & f-strings

```
text = "22,80,443"
text[0]          # '2' (index one character)
text[0:2]        # '22' (slice a piece)
text.split(",")  # ['22', '80', '443']

p = 80
print(f"Port {p} is open")  # f-string plugs values in
```

An **f-string** lets you drop a variable straight into text with `{ }`.

 `"22"` (text) is **not** `22` (number) — this matters for ports later!

Lists & dictionaries

```
ports = [21, 22, 80, 443]      # ordered list
ports.append(8080)             # add to it
ports[0]                       # 21

services = {22: "ssh", 80: "http"} # key: value pairs
services[22]                   # "ssh"
services.get(443, "unknown")   # safe lookup w/ fallback
```

- **List** `[ ]` — an ordered collection.
- **Dictionary** `{ }` — look things up by key.

Conditionals & loops

```
for port in ports:
    if port == 22:
        print(f"{port} is SSH")
    elif port == 80:
        print(f"{port} is HTTP")
    else:
        print(f"{port} is something else")
```

- `if` / `elif` / `else` — branch on a test.
- `for` loops over a collection; `while` loops while a condition holds.
- A port scanner is really just: **loop a list of ports → decide open or closed.**

Functions

```
def check_port(ip, port):  
    # ... do the work ...  
    return True          # hand a result back  
  
if check_port("192.168.56.10", 22):  
    print("open!")
```

- `def` defines a reusable, named block.
- **Parameters** (`ip`, `port`) are inputs; `return` hands a value back.
- Define once, call as many times as you like.

The socket module & try/except

```
import socket

def check_port(ip, port):
    s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    s.settimeout(0.5)           # don't hang on closed ports
    try:
        result = s.connect_ex((ip, port)) # 0 == OPEN
        return result == 0
    except socket.error:
        return False
    finally:
        s.close()
```

Ethics & Authorization

Writing the tool yourself does not change the law.

A port scanner is a **real scanning tool** — building it in Python instead of running `nmap` makes no legal difference.

Port scanning a host you don't own or have written permission to test can violate the **CFAA** and state law.

The only legal target this week: the single isolated host-only lab target.
Confirm with `ip addr` *before every* network run.

Key vocabulary

Term	Meaning
Interpreter / REPL	<code>python3</code> runs code / live one-line prompt
Type	<code>str</code> , <code>int</code> , <code>float</code> , <code>bool</code>
f-string	Text with values plugged in: <code>f"Port {p}"</code>
List / Dictionary	<code>[22, 80]</code> / <code>{22: "ssh"}</code>
Function	Reusable block via <code>def</code> , takes inputs, <code>return</code> s
Module	Ready-made code you <code>import</code> (e.g., <code>socket</code>)
Port / Timeout	A service's numbered door / limit before giving up
Exception / <code>try</code> - <code>except</code>	A runtime error / catch it and keep going

Lab launch: build a port scanner

Platform: your **Kali VM** on the **host-only / isolated** lab network.

First, every day:

```
ip addr          # confirm you're on the lab subnet (192.168.56.x)
python3 --version # Python 3.x.x
mkdir -p ~/unit06 && cd ~/unit06
```

You'll build, in parts:

1. **hello.py** — operator banner (variables, `input()`, f-strings)
2. **scanner.py data** — the `ports` list + `services` dictionary
3. **scanner.py scan** — `check_port()` + loop → list open ports, save to file

The scanner, assembled

```
target = input("Lab target IP to scan: ")
print(f"Scanning {target} (lab target only) ...")

open_ports = []
for port in ports:
    if check_port(target, port):
        name = services.get(port, "unknown")
        print(f"  [+] {port}/tcp open  ({name})")
        open_ports.append(port)

print(f"Done. {len(open_ports)} open port(s): {open_ports}")
```

15

Recap

- Python runs **interactively** or as a **script** — `python3`, indentation is syntax.
- **Types** matter: `"22"` (str) $\neq$ `22` (int). Convert with `int()` / `str()`.
- **Lists** `[ ]` and **dictionaries** `{ }` hold your data.
- **Functions** (`def` ... `return`) + `if` / `for` build the logic.
- `socket` + **timeout** + `try` / `except` = a reliable scanner.
- Same skills as Bash week, new language — and **authorization still rules.**

Exit ticket & discussion

1. Name the two ways to run Python and one difference.
2. What does `connect_ex` returning `0` mean — and why use a timeout?
3. What does `try/except` let your program do when an error happens?

Discuss: Name one thing your scanner does that makes it **reliable** — and one place it would be **illegal** to run it.

Submit `hello.py`, `scanner.py`, a screenshot running, your open-port list, and `scan_results.txt`.